

Tool-Augmented Reasoning with Group Relative Policy Optimization

Final Project Report
Course: Advanced AI

Yueyang Cheng	Student ID: 2025245047	<i>Algorithm Design & RL Training</i>
Wenzhe Dong	Student ID: 2025245048	<i>SFT Pipeline & Sandbox Infrastructure</i>
Jian Lv	Student ID: 2025245046	<i>Data Engineering & Baseline Experiments</i>
Xin Chen	Student ID: 2025245045	<i>Literature Survey & Result Analysis</i>
Haoyu Lu	Student ID: 2025245042	<i>Evaluation Framework & Visualization</i>

May 8, 2026

Abstract

Large language models (LLMs) exhibit remarkable reasoning capabilities, yet they remain fundamentally limited by their inability to perform precise numerical computation or verify intermediate reasoning steps. We present **ReTool**, a reinforcement learning framework that trains LLMs to autonomously invoke a Python code interpreter as a tool during multi-step mathematical reasoning. Our approach consists of two phases: (1) a *cold-start* supervised fine-tuning (SFT) stage that teaches the model the basic protocol of code generation and execution-feedback interpretation, and (2) a reinforcement learning stage built on **Group Relative Policy Optimization** (GRPO) with **asymmetric clipping** and a novel **tool-usage reward shaping** mechanism. We evaluate on four mathematical benchmarks spanning elementary to competition level: GSM8K, MATH-500, AIME 2024, and AIME 2025. Our results show that applying GRPO directly on the base instruct model yields limited performance (AIME 2024 pass@16 = 4.19%), while adding a tool-specific SFT warm-up phase lifts this to 19.52%. Subsequent GRPO training on top of the SFT initialization further improves MATH-500 to 73.17% and AIME 2024 to 20.28%. We identify an interesting degradation pattern on AIME 2025 and provide a detailed analysis of the trade-offs involved in KL-constrained vs. unconstrained policy optimization.

1 Introduction

Large language models have demonstrated impressive performance on a wide range of reasoning benchmarks. However, when faced with mathematical problems that require precise multi-step computation—solving equations numerically, evaluating trigonometric expressions, or iterating through candidate solutions—pure text-based reasoning often falls short. Unlike humans, who naturally reach for calculators, scratch paper, or programming environments, LLMs must rely entirely on their internalized knowledge, which is prone to arithmetic errors and cumulative drift over long reasoning chains.

A natural remedy is to equip LLMs with external tools, such as a Python interpreter, and allow them to interleave reasoning with code execution. Prior work has explored tool-augmented LLMs through prompting or supervised fine-tuning, but these approaches suffer from a *degeneration* problem: without explicit incentives, models tend either to over-use tools (calling the interpreter unnecessarily for trivial steps) or to under-use them (attempting to solve everything through generation).

In this project, we design a reinforcement learning framework that lets the model *learn when and how to use a code execution tool* through trial, error, and reward. Our key contributions are:

1. **GRPO-based RL with asymmetric clipping.** We replace the standard PPO algorithm with Group Relative Policy Optimization, removing the need for a critic network and enabling stable training on binary-outcome math tasks. Asymmetric clipping (0.2 low / 0.38 high) allows the model to make larger updates on the rare occasions when it discovers correct reasoning paths.
2. **Tool-usage reward shaping.** We introduce a reward adjustment that softens the penalty for incorrect answers when the model engages in more tool-calling turns, encouraging deeper exploration and iterative debugging rather than shallow guesswork.
3. **Two-stage training pipeline.** A cold-start SFT phase teaches the model the basic protocol of code generation and interpreter interaction, followed by GRPO reinforcement learning that refines the model’s strategic tool-use decisions.
4. **Comprehensive empirical analysis.** We benchmark across four datasets at progressively higher difficulty levels, providing insight into when and why RL helps (or hurts) tool-augmented reasoning.

2 Related Work

2.1 LLM Reasoning and Tool Use

Recent work has shown that LLMs can leverage external tools—search engines, calculators, code interpreters, and APIs—to extend their capabilities beyond pure text generation. Tool-augmented LLMs such as Toolformer and ToolLLM demonstrate that tool-calling can be learned through supervised fine-tuning on annotated interaction traces. However, SFT-based approaches rely on the quality and coverage of the demonstration data, and models trained this way lack the ability to strategically decide *when* tool use is beneficial versus when it is unnecessary overhead.

2.2 Reinforcement Learning for LLMs

Reinforcement Learning from Human Feedback (RLHF) and its variants have become the standard paradigm for aligning LLMs with desired behaviors. Proximal Policy Optimization (PPO) is widely used but requires maintaining a separate critic (value) network, which substantially increases memory footprint. More recently, Group Relative Policy Optimization (GRPO) was proposed as a simpler alternative: for each prompt, multiple responses are sampled and their rewards are normalized within the group to produce an advantage signal, eliminating the need for a learned value function. Variants such as DAPO (Decoupled-clipped Alignment Policy Optimization) further refine GRPO by removing the KL-divergence constraint from the reward, enabling more flexible exploration.

2.3 Code-Augmented Mathematical Reasoning

Tool-integrated reasoning for mathematics has been explored in systems like PAL (Program-Aided Language models) and PoT (Program of Thoughts), where models generate executable code to solve math problems. These approaches typically rely on few-shot prompting with fixed examples. In contrast, our work trains the model end-to-end via RL to internalize the *strategy* of tool use, learning not only how to write correct code but also how to interpret execution feedback and decide whether to continue iterating or produce a final answer.

3 Methodology

Our training pipeline starts from the Qwen2.5-3B-Instruct base model and proceeds through two additional stages: (1) a *tool-specific* supervised fine-tuning (SFT) phase on multi-turn code-execution trajectories, serving as a cold-start warm-up, and (2) GRPO-based reinforcement learning that optimizes the model’s strategic decisions about when and how to invoke the code interpreter. To quantify the contribution of each stage, we compare three configurations: (i) GRPO applied directly to the base instruct model (skipping tool-specific SFT), (ii) tool-specific SFT only (no RL), and (iii) the full pipeline of tool-specific SFT followed by GRPO.

3.1 Cold-Start Supervised Fine-Tuning (SFT)

Directly applying GRPO to the base Qwen2.5-3B-Instruct model—without any tool-specific SFT warm-up—fails to produce effective tool-use behavior. The model, though generally capable from its instruction tuning, has never seen the specific format of multi-turn code execution and interpreter feedback. It produces syntactically invalid code blocks, fails to format answers with the required `\boxed{}` notation, or ignores the tool altogether and generates pure-text guesses. Under GRPO, if all N sampled responses for a prompt receive the minimum reward (-1), the group-normalized advantage is identically zero, and the policy receives no learning signal. This *cold-start problem* motivates an explicit tool-specific SFT warm-up phase.

We address this by first fine-tuning the model on multi-turn code-execution trajectories. Each trajectory is a sequence of alternating messages:

`user → assistant (code) → tool (execution output) → assistant (answer)`

The SFT data is constructed by parsing raw interaction traces containing `<code>`, `<interpreter>`, and `<answer>` tags, and converting them into OpenAI-compatible multi-turn message format with proper `tool_calls` schema. We use the Qwen2.5-3B-Instruct base model and train for 6 epochs with a cosine learning rate schedule (peak LR = 1×10^{-5} , warmup = 5%), max sequence length = 16,384 tokens.

3.2 GRPO Reinforcement Learning

After SFT, the model can reliably format code blocks and interpret execution results. We then apply GRPO to train the model to make *strategic* decisions: how many turns of tool calling to use, when to trust an execution result, and when to run additional verification.

3.2.1 Advantage Estimation via Group Normalization

For each prompt x_i , we sample $N = 8$ responses $\{y_i^1, y_i^2, \dots, y_i^N\}$ from the current policy π_θ . Each response is scored via our reward function $R(y_i^j)$, yielding rewards $\{r_i^1, \dots, r_i^N\}$. The advantage for response j is computed by group-wise standardization:

$$\hat{A}_i^j = \frac{r_i^j - \text{mean}(\mathbf{r}_i)}{\text{std}(\mathbf{r}_i) + \epsilon} \quad (1)$$

Intuitively, Equation 1 means that within each group of 8 attempts at the same problem, responses that perform *above average* receive a positive advantage and are reinforced, while below-average responses are suppressed. This internal competition mechanism naturally stabilizes training: the scale of the advantage adapts to the reward variance within each group, and no separate value function needs to be learned.

3.2.2 Asymmetric Clipping

Standard PPO clips the policy ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ to $[1 - \epsilon, 1 + \epsilon]$ with $\epsilon = 0.2$. We modify this to use asymmetric bounds:

$$L^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \hat{A}_t \right) \right] \quad (2)$$

with $\epsilon_{\text{low}} = 0.2$ and $\epsilon_{\text{high}} = 0.38$.

The motivation is task-specific: in mathematical reasoning, correct solutions are sparse. Most sampled attempts fail (advantage < 0), but the rare correct attempt (advantage > 0) carries disproportionately valuable information. A higher upper clip bound lets the policy take larger steps toward these successful strategies when they are discovered, while the conservative lower bound protects against the much more common negative-advantage updates. This design embodies the intuition: *be aggressive with success, be cautious with failure*.

3.2.3 Tool-Usage Reward Shaping

Our base reward function uses exact-answer matching with `\boxed{\}` extraction: +1 for a correct answer, −1 for incorrect. A naive binary reward, however, treats “used tools for 10 turns but made a minor mistake” and “did not attempt the problem at all” identically. This discourages the model from using the code interpreter, since each additional tool call consumes tokens and risks syntax errors without any compensating incentive.

We introduce a turn-dependent penalty softening mechanism:

$$R_{\text{adjusted}} = \begin{cases} +1, & \text{if answer is correct} \\ \min(-0.6, -1 + \frac{(\text{num_turns}-2)}{2} \times 0.1), & \text{if answer is incorrect} \end{cases} \quad (3)$$

The formula works as follows: the minimum number of turns required to produce an answer is 2 (one code generation + one final response). At 2 turns, the penalty remains the full −1.0. Each additional turn beyond the minimum adds a +0.05 softening, up to a cap of −0.6. This ensures the model is never penalized *less* than −0.6, preventing degenerate behavior where the model endlessly calls tools to accumulate reward without ever producing a correct answer.

This reward shaping communicates a nuanced training signal: *failing after serious effort is preferable to failing without trying*.

3.2.4 Training Configuration

Table 1 summarizes the key RL training hyperparameters.

Table 1: GRPO training configuration.

Parameter	Value	Description
Base model	Qwen2.5-3B-Instruct (SFT checkpoint)	Initialized from cold-start
Training data	DAPO-Math-17k	~17,000 competition-level math problems
Samples per prompt (N)	8	GRPO group size
Max tool-calling turns	16	Upper bound on code invocation rounds
Max response length	16,384 tokens	Per-turn generation budget
Actor learning rate	1×10^{-6}	Low LR for stable policy updates
PPO mini-batch size	2	Gradient accumulation granularity
Train batch size	4	Prompts per training step
Total epochs	1	Single pass over training data
Clip ratio (low / high)	0.2 / 0.38	Asymmetric bounds
KL penalty	Disabled (coef = 0)	DAPO-style unconstrained exploration
Rollout engine	SGLang (async mode)	High-throughput sampling
Hardware	2× NVIDIA A100-80GB	FSDP with CPU offloading

3.3 Code Execution Sandbox

A robust and secure code execution environment is essential infrastructure for this project. We built a standalone FastAPI-based sandbox server with the following safety mechanisms:

- **OS resource limits:** `setrlimit` caps virtual memory (256 MB), CPU time (5 s wall, 10 s total), process count (max 8), and output file size (10 MB).
- **Privilege restriction:** `prctl(PR_SET_NO_NEW_PRIVS, 1)` prevents any form of privilege escalation.
- **System call filtering:** An optional `seccomp` filter blocks dangerous syscalls (`execve`, `socket`, `connect`, `bind`, `ptrace`, `mount`, `chroot`, `unshare`).
- **Memory monitor:** A background coroutine polls RSS every 50 ms and hard-kills the subprocess on OOM.
- **Concurrency control:** An async semaphore caps simultaneous executions at 16.
- **Ephemeral workspace:** Each execution gets a unique temporary directory, cleaned up in a `finally` block regardless of exit condition.

Together, these measures ensure that even if the model generates malicious code (e.g., `os.system("rm -rf /")`), the host system is protected by OS-level isolation.

4 Experimental Setup

4.1 Datasets

We evaluate on four mathematical reasoning benchmarks spanning a wide difficulty range:

Table 2: Evaluation benchmarks.

Dataset	Domain	Difficulty	# Problems
GSM8K	Grade-school math	Elementary	1,319
MATH-500	High-school competition	Intermediate	500
AIME 2024	American Invitational Mathematics Exam	Advanced	30
AIME 2025	American Invitational Mathematics Exam	Advanced	30

4.2 Training Paradigms Compared

We compare three configurations that isolate the contribution of each training stage:

1. **Qwen 3B Instruct:** GRPO applied directly to Qwen2.5-3B-Instruct *without* the tool-specific SFT warm-up. This tests whether RL alone can discover tool-use strategies from scratch.
2. **Tool-Specific SFT Only:** The base instruct model further fine-tuned on multi-turn code-execution trajectories. This serves as the cold-start initialization for RL, but here we evaluate it without subsequent RL to measure the standalone effect of tool-specific SFT.
3. **Tool SFT + GRPO:** Our full pipeline: base instruct $\rightarrow$ tool-specific SFT $\rightarrow$ GRPO reinforcement learning.

4.3 Evaluation Metrics

We report **pass@ k** : for each problem, we sample k independent responses ($k \in \{2, 4, 8, 16\}$). The problem is considered solved if *any* of the k responses produces the correct answer. Formally, let $c_i \in \{0, 1\}$ indicate correctness of the i -th sample; then $\text{pass}@k = \mathbb{E}_{\text{problems}} \left[1 - \prod_{i=1}^k (1 - c_i) \right]$, estimated empirically. This metric is particularly relevant for tool-augmented systems, where multiple solution strategies may be explored across different samples.

All responses are generated with temperature = 1.0 and nucleus sampling with top- p = 0.8 to ensure diversity.

5 Results and Analysis

5.1 Main Results

Tables 3–6 present the full benchmark results across all three training configurations.

Table 3: GSM8K results (pass@ k , %).

Configuration	pass@2	pass@4	pass@8	pass@16
Qwen 3B Instruct	65.63	80.72	89.56	93.57
Tool-Specific SFT Only	84.22	91.91	94.99	96.43
Tool SFT + GRPO	84.32	92.87	95.95	96.92

Table 4: MATH-500 results (pass@ k , %).

Configuration	pass@2	pass@4	pass@8	pass@16
Qwen 3B Instruct	41.46	50.06	56.52	60.72
Tool-Specific SFT Only	50.82	59.83	66.04	69.92
Tool SFT + GRPO	49.85	60.78	68.44	73.17

Table 5: AIME 2024 results (pass@ k , %).

Configuration	pass@2	pass@4	pass@8	pass@16
Qwen 3B Instruct	0.75	1.41	2.64	4.19
Tool-Specific SFT Only	5.65	9.27	14.06	19.52
Tool SFT + GRPO	8.62	12.46	16.51	20.28

Table 6: AIME 2025 results (pass@ k , %).

Configuration	pass@2	pass@4	pass@8	pass@16
Qwen 3B Instruct	1.15	2.09	3.45	5.03
Tool-Specific SFT Only	4.17	6.05	7.62	8.82
Tool SFT + GRPO	2.83	4.73	7.53	9.64

5.2 Key Findings

Finding 1: Tool-specific SFT warm-up is essential. On AIME-level problems, applying GRPO directly to the base instruct model—without tool-specific SFT—achieves only 0.75% pass@2 (AIME 2024) and 1.15% (AIME 2025). The model cannot learn effective tool-use behaviors from scratch because the initial policy generates nearly zero correct responses, leading to zero-variance advantage estimates and no gradient signal. Adding a tool-specific SFT phase before RL provides the necessary behavioral bootstrap.

Finding 2: Tool-specific SFT alone is surprisingly strong. Adding tool-specific SFT on top of the base instruct model lifts AIME 2024 pass@16 from 4.19% to 19.52%—nearly a $5\times$ improvement—and achieves 96.43% on GSM8K. This indicates that carefully constructed multi-turn code-execution trajectories contain rich reasoning knowledge that can be effectively transferred through supervised learning, even without any RL.

Finding 3: GRPO further refines tool-use strategy, with larger gains on harder problems. On GSM8K (already 96.43% with tool SFT alone), the subsequent GRPO stage adds only 0.49 points (96.92%). However, on MATH-500, the gap widens from 69.92% to 73.17% at pass@16. On AIME 2024, the full pipeline achieves 20.28% vs. 19.52% (tool SFT only). The RL stage appears most beneficial on problems that require *strategic* tool use—deciding how many verification rounds to run, or whether to try an alternative approach after a failed code execution.

Finding 4: Anomalous degradation on AIME 2025. AIME 2025 is the only benchmark where the full pipeline (Tool SFT + GRPO) underperforms Tool-Specific SFT Only at low sample budgets (pass@2: 2.83% vs. 4.17%). We hypothesize two contributing factors:

1. **Distribution shift:** The RL training data (DAPO-Math-17k) overlaps more with AIME 2024 than AIME 2025 in problem style. While AIME 2024 was used as a validation set during training, AIME 2025 serves as a truly out-of-distribution test, revealing that RL-induced policy changes do not always generalize.
2. **Catastrophic forgetting:** With KL penalty disabled, the policy can drift substantially from the SFT initialization. On problems where the SFT model already had a reasonable strategy, RL may overwrite it with a noisier one optimized for the training distribution.

Finding 5: pass@ k scaling reveals latent capability. Across all benchmarks and configurations, pass@ k improves substantially as k increases from 2 to 16—often nearly doubling. For example, the full pipeline (Tool SFT + GRPO) on MATH-500 goes from 49.85% (pass@2) to 73.17% (pass@16). This scaling indicates that the model has learned *diverse* solution strategies rather than converging to a single mode; sampling more responses increases the chance that at least one captures a correct reasoning path.

5.3 Ablation: Effect of Asymmetric Clipping

We conducted a targeted ablation comparing symmetric clipping ($\epsilon = 0.2$) against our asymmetric setting ($\epsilon_{\text{low}} = 0.2$, $\epsilon_{\text{high}} = 0.38$). While a full parameter sweep was beyond our computational budget, the asymmetric variant consistently produced higher pass@16 scores across all benchmarks, suggesting that the larger update steps on positive-advantage samples are beneficial for this sparse-reward domain.

6 Discussion

6.1 Why Remove the KL Constraint?

Our GRPO configuration disables both the KL penalty in the reward and the KL loss term in the actor objective. This follows the DAPO design philosophy: the KL constraint prevents the policy from deviating too far from a reference model, but it also limits exploration. In tool-use tasks, the tool-specific SFT model already provides a reasonable initialization, and the main improvements come from discovering *new* strategies—e.g., learning to add a verification round when the first code execution produces an ambiguous result. Removing the KL constraint gives the policy the freedom to make these qualitative behavioral leaps.

The downside, as AIME 2025 suggests, is reduced robustness to distribution shift. A promising direction for future work is a *curriculum-based* KL schedule: strong KL constraints early in training to prevent catastrophic forgetting, gradually relaxed as the policy accumulates stable tool-use patterns.

6.2 Design Rationale of Tool Reward Shaping

The reward softening formula (Eq. 3) encodes a deliberate pedagogical principle: *process matters, not just outcomes*. In human education, a student who works through a problem methodically but makes an arithmetic error is graded more leniently than one who submits a random guess. Our reward shaping operationalizes this intuition for LLM training.

The specific values (-0.6 cap, $0.05/\text{turn}$ slope) were determined through iterative tuning. A cap that is too high (e.g., -0.1) would incentivize the model to maximize turn count without pursuing correctness. A cap that is too low (e.g., -0.9) would provide negligible differentiation from the unmodified -1 . We found -0.6 to provide a balanced incentive for exploration without enabling reward gaming.

6.3 Limitations

1. **Model scale.** All experiments use Qwen2.5-3B-Instruct. While 3B parameters suffice for learning tool-use protocols, larger models (7B, 14B, 72B) would likely exhibit qualitatively better reasoning, especially on AIME-level problems.
2. **Single tool type.** Our sandbox only supports Python code execution. Extending to symbolic mathematics (SymPy), web search, or geometry tools could unlock capabilities on a broader range of problems.
3. **Offline RL.** Training uses a fixed dataset (DAPO-Math-17k). An online RL setup, where the model continuously generates and learns from new problems, could produce more robust policies.
4. **Limited compute for ablation.** A thorough understanding of the interaction between clip ratios, KL coefficients, and reward shaping hyperparameters would require systematic grid searches that exceed our current computational resources.

7 Conclusion

We presented ReTool, a training framework that teaches LLMs to strategically use a Python code interpreter for mathematical reasoning. Starting from Qwen2.5-3B-Instruct, a tool-specific SFT phase establishes basic code-execution competence, and a subsequent GRPO phase—augmented with asymmetric clipping and tool-usage reward shaping—refines the model’s decision-making about when and how intensively to invoke tools. Our experiments on four benchmarks demonstrate that the full pipeline substantially outperforms applying GRPO without the tool-specific warm-up, achieving 73.17% pass@16 on MATH-500 and 20.28% on AIME 2024.

The degradation observed on AIME 2025 highlights an important open question in tool-augmented RL: how to balance exploration (enabled by removing KL constraints) with out-of-distribution robustness. We believe this tension is a fruitful direction for future investigation, with potential applications beyond mathematical reasoning to any domain where strategic tool use enhances LLM capabilities.

8 Team Contributions

Table 7: Individual contributions of team members.

Member	Contributions
Yueyang Cheng	• Designed the overall algorithm architecture: GRPO with asymmetric clipping and tool-usage reward shaping • Implemented the GRPO reinforcement learning training pipeline, including custom reward function, dataset class, and tool execution wrapper • Configured and executed all RL training runs on GPU cluster • Conducted hyperparameter tuning for clip ratios, reward shaping parameters, and learning rate schedules • Led the analysis of experimental results and authored the technical sections of this report
Wenzhe Dong	• Implemented the data preprocessing pipeline for SFT trajectories, converting raw interaction traces to OpenAI-compatible multi-turn message format • Developed the code execution sandbox server with OS-level resource isolation and security mechanisms • Conducted SFT training runs and performed checkpoint selection for downstream RL initialization
Jian Lv	• Curated and preprocessed training and evaluation datasets (GSM8K, MATH-500, AIME 2024/2025, DAPO-Math-17k) • Ran the Qwen 3B Instruct baseline experiments (GRPO directly on the instruct model, without tool-specific SFT warm-up) • Implemented the evaluation framework supporting pass@k metric computation across multiple sampling budgets
Xin Chen	• Conducted literature survey on LLM tool use, PPO/GRPO variants, and code-augmented reasoning • Compared and summarized related work, helping to position the project within the research landscape • Performed statistical analysis of experimental results and contributed to the discussion of findings
Haoyu Lu	• Implemented FSDP checkpoint conversion and HuggingFace-compatible model export • Set up experiment logging and monitoring infrastructure (SwanLab integration) • Generated data visualizations and result plots for this report and the presentation